

Fixed-Point OMP for ECG Compressed Sensing: a precision-resource study on the Zynq-7020

Brandon Carido, Michael Dang, Connor Machado
 ENGS 109: High-dimensional Sensing and Learning (HdSL)
 Dartmouth College

Abstract

Compressed sensing reduces the data a wearable ECG sensor must acquire and transmit, but shifts the cost to signal recovery, which must run cheaply on embedded hardware. We implement a fixed-point Orthogonal Matching Pursuit (OMP) recovery engine for the Zynq-7020, partitioning the algorithm so the parallel correlation step maps to programmable logic while the sequential least-squares solve runs in software on the Cortex-A9. Using a Bernoulli ± 1 measurement matrix at a 0.5 compression ratio ($M = 128$, $N = 256$) and a Daubechies-4 wavelet sparsifying basis, we characterize the design across MAC operand bit-widths on real MIT-BIH ECG data. Our central result is a precision–resource Pareto front: reconstruction error (PRD) is essentially flat from 16 bits down to 8 bits, then degrades sharply below 6 bits, identifying 11-bit arithmetic as the efficient operating point. Below 11 bits the multipliers consume LUT fabric; at 11 bits they map to the otherwise-idle DSP array, freeing fabric for the AXI interface, control, and other system logic at no accuracy cost. Against a tuned Cortex-A9 software baseline (187 μ s per correlation pass), the fixed-point engine achieves an estimated 18 $\times$ speedup on the correlation step.

Index Terms

Orthogonal Matching Pursuit, sparse signal recovery, compressed sensing, FPGA acceleration, Zynq-7000 SoC, hardware/software co-design, Vivado synthesis, resource utilization, embedded systems

I. INTRODUCTION

Orthogonal Matching Pursuit is a greedy sparse approximation algorithm that iteratively selects dictionary atoms based on their correlation with the current residual. The broader matching pursuit framework was introduced by Mallat and Zhang for decomposing signals over time-frequency dictionaries [1]. Tropp and Gilbert later analyzed Orthogonal Matching Pursuit in the compressed sensing setting, showing how sparse signals can be recovered from random measurements under appropriate conditions [2]. This project targets the correlation step in Orthogonal Matching Pursuit (OMP), a sparse recovery algorithm used here in a compressed-sensing ECG reconstruction flow. The goal is to accelerate the repeated dictionary-residual correlation on FPGA fabric while leaving the least-squares solve, support management, and residual update in software. That split matches the structure of OMP: for an $M \times N$ dictionary, each iteration requires N inner products of length M , which makes correlation the most regular and parallel part of the algorithm.

The checked-in system parameters are a measurement length of $M = 128$, $N = 256$ dictionary atoms, signal windows of length 256, and a software-enforced support cap of 32 iterations. Hardware-visible inputs use signed $Q1.15$ fixed-point format at a 100 MHz target clock, while an internal `MAC_BITS` sweep is used to study the precision-resource tradeoff for the correlation engine.

II. BACKGROUND

OMP approximates a measurement vector $y \in \mathbb{R}^M$ with a small number of dictionary atoms from $D \in \mathbb{R}^{M \times N}$. At iteration t , the atom-selection step is

$$\lambda_t = \arg \max_j |D_j^T r_t|$$

where r_t is the current residual. After selecting a support set S_t , software solves

$$\hat{x}_{S_t} = \arg \min_z \|y - D_{S_t} z\|_2$$

and updates the residual as

$$r_{t+1} = y - D_{S_t} \hat{x}_{S_t}.$$

The FPGA engine is responsible only for the correlation kernel,

$$a_j = \sum_{i=0}^{127} D_{i,j} r_i,$$

followed by an argmax over $|a_j|$. This partition is attractive because correlation is dominated by multiply-accumulate work and reduction logic, while the least-squares step is more control-heavy and numerically irregular.

The fixed-point interface uses Q1.15 inputs, so multiplying two inputs produces Q2.30 products and summing 128 terms requires additional accumulator headroom. The software models therefore use wider integer accumulation to avoid overflow during reference evaluation. Tie behavior is defined as "lowest index wins," matching 'numpy.argmax' and the Cortex-A9 baseline's strict ' $\leq$ ' comparison. Reconstruction quality is reported with percent root-mean-square difference (PRD):

$$\text{PRD} = 100 \frac{\|x - \hat{x}\|_2}{\|x\|_2}.$$

III. METHODS

A. Software simulation to determine accuracy and which MAC_BITS perform best

The software accuracy flow combines the floating-point and fixed-point reference code in `SW/omp_reference.py` with the HDL-side simulator notebook `HDL-B/omp_accuracy_characterization.ipynb`. The floating-point OMP implementation establishes an accuracy ceiling, while `hw_correlate_q15()` and `omp_argmax_fixed()` model the hardware correlation step exactly: dictionary and residual values are quantized to Q1.15, optionally truncated according to `MAC_BITS`, accumulated with integer arithmetic, and compared by absolute correlation with lowest-index tie breaking.

The notebook documents two dictionary/basis stages. It first builds a compressed-sensing dictionary $D = \Phi\Psi$ and then records a later "final summary" note that the sparsifying basis was switched from DCT-II to an orthonormal db4 wavelet basis. That notebook note explicitly states that the change lowered the reconstruction floor from an unusable regime of about 57% PRD to a usable regime near 20% PRD, which made the bit-width tolerance study more meaningful.

Verification was layered. Synthetic tests included an 8-sparse recovery case with exact support recovery and a reported PRD of 0.003%. Real-data evaluation used MIT-BIH ECG windows in the notebook, with OMP loops run at different support sizes and `MAC_BITS` settings. The same flow also validated five reusable golden-vector cases stored under `goldenVectors`: an easy margin case, a tight margin case, a winner near index 0, a winner near index $N - 1$, and an engineered tie. This structure allowed the project to study how reduced internal precision changes support selection without changing the external Q1.15 interface.

B. Software simulation on the Cortex-A9 to measure OMP correlation speed

The software baseline in `SW/processor_simulation/omp_baseline.c` times only `correlation_pass()`, which mirrors one hardware compute pass by correlating the current residual against all 256 atoms and returning the argmax of the absolute inner product [1], [2]. The rest of the OMP loop remains in software and is left untimed so the baseline can still generate reconstructed signals and PRD values for cross-checking.

Two timing contexts are documented in the repo and should be kept separate. First, `FinalReport/process_sim_notebook.ipynb` records a host-side baseline run over 120 windows and 3840 correlation passes, with a mean PRD of 20.1318% and a mean per-pass time of 19.0713 μ s. Second, the bare-metal Zynq timing path in `SW/VitisWorkspace/processor_baseline/src/main_zynq.c` reports a tuned compute-only Cortex-A9 measurement of about 187.124 μ s per correlation pass over 1000 repetitions, at Cortex-A9 @ 667 MHz with `-O3 -mfpu=neon -mfloat-abi=hard -ffast-math`; that measurement explicitly excludes AXI transfer overhead.

The checked-in `baseline_results.csv` covers 120 reconstructed windows with 32 selected atoms per window. From that file, the PRD summary is: count 120, mean 20.1318%, median 16.5762%, minimum 9.0077%, and maximum 80.8797%. Those values quantify algorithmic reconstruction behavior, while the timed per-pass numbers quantify correlation-kernel cost.

C. Hardware simulation on the programmable logic for synthesis, timing, and latency

Hardware characterization is driven by the Vivado 2021.2 synthesis sweep captured in `HDL-A/resource_sweep.csv` and parsed by `HDL-A/parse_util.py`. The sweep covers `MAC_BITS = 3, 4, 5, 6, 8, 10, 11, 12, 16` and records slice LUTs, DSPs, BRAM tiles, worst negative slack (WNS), and an estimated maximum frequency derived from the 10 ns target period [3]. Across all checked-in sweep points, BRAM usage is constant at 16 tiles, indicating that storage dominates BRAM cost more than arithmetic width does.

TABLE I: Vivado 2021.2 Resource Sweep Results

MAC_BITS	LUT	DSP	BRAM	WNS (ns)	Fmax (MHz)
3	867	0	16	2.780	138.5
4	1473	0	16	2.780	138.5
5	1760	0	16	2.780	138.5
6	2078	0	16	2.780	138.5
8	3034	0	16	2.780	138.5
10	4503	0	16	1.162	113.1
11	610	47	16	2.780	138.5
12	610	47	16	2.780	138.5
16	610	47	16	2.780	138.5

The main architectural crossover occurs at 11 bits: up through 10 bits the design remains LUT-based and LUT cost rises with precision, but at 11 bits and above Vivado infers 47 DSP blocks and LUT count drops sharply to 610. All listed sweep points still meet the 100 MHz target. Based on the checked-in sweep, `MAC_BITS = 11` is the clearest operating point because it preserves timing margin while substantially reducing LUT pressure.

For latency, the report includes one value that is not repo-derived: a user-supplied estimate of 1033 cycles at 100 MHz for the correlation engine alone. That corresponds to 10.33 μ s if converted directly from cycles and clock period, but it is included here as a user-provided hardware result rather than a measurement extracted from `resource_sweep.csv`.

IV. REFLECTION

The strongest outcome of the project is not a full end-to-end hardware speedup claim, but a well-bounded study of the OMP correlation kernel. Restricting hardware acceleration to correlation kept the interface simple, made tie behavior and fixed-point correctness testable with deterministic golden vectors, and avoided moving the least-squares solve into fabric before the precision story was understood. The results also show that hardware cost does not decrease monotonically with reduced bit width: the 11-bit point is cheaper in LUTs than the 10-bit point because synthesis changes multiplier mapping. At the same time, the software PRD spread shows that reconstruction quality is limited by the sparse model and support selection behavior, not only by arithmetic precision.

V. FUTURE WORK

The next step is hardware-in-the-loop measurement that includes AXI communication overhead rather than only synthesis timing and software compute baselines. The repo also makes it clear that hardware latency should be documented and validated in checked-in artifacts instead of only being carried as an external note. After that, DMA or batched transfers would be natural improvements for reducing data-movement cost, and a later version could consider accelerating more of the OMP loop, such as least-squares support updates, once the correlation-offload path is fully characterized. Broader multi-record validation would also help separate dictionary limitations from fixed-point implementation effects.

REFERENCES

- [1] S. G. Mallat and Z. Zhang, “Matching pursuits with time-frequency dictionaries,” *IEEE Transactions on Signal Processing*, vol. 41, no. 12, pp. 3397–3415, Dec. 1993.
- [2] J. A. Tropp and A. C. Gilbert, “Signal recovery from random measurements via orthogonal matching pursuit,” *IEEE Transactions on Information Theory*, vol. 53, no. 12, pp. 4655–4666, Dec. 2007.
- [3] Xilinx, *Zynq-7000 SoC Data Sheet: Overview*, Xilinx, Jul. 2018, product Specification.